

Google Summer of Code 2026 — ML4SCI

Learning Quantum Representations of Classical High Energy Physics Data with Contrastive Learning

Project: QMLHEP6

Abhipsa Acharya

PhD Student, Physics and Machine Learning
aviiacharya@github

Mentors: Rui Zhang, Alkaid Cheng, Sergei Gleyzer, Emanuele Usai, Tom Magorsh
University of Wisconsin-Madison | University of Alabama | TUM

1. Abstract

I propose to develop trainable quantum embedding functions for encoding classical high energy physics (HEP) data into quantum representations using contrastive learning. The architecture is a hybrid: a PointVector encoder (a point cloud network with directional attention) compresses each particle jet into a 256-dimensional vector, a linear layer maps this to 64 dimensions, and a 6-qubit Variational Quantum Circuit (VQC) processes the result through amplitude embedding, ZXZ single-qubit rotations (RZ-RX-RZ, a standard decomposition of any single-qubit unitary), and Controlled-NOT (CNOT) entangling gates in ring and skip-connection topologies. The measurement returns a 64-dimensional probability vector. Training uses SimCLR (Simple Framework for Contrastive Learning of Visual Representations), which creates two augmented views of each jet, encodes both, and optimizes the NT-Xent loss (Normalized Temperature-scaled Cross-Entropy) to pull views of the same jet together and push different jets apart.

My working prototype on the Pythia8 Quark-Gluon dataset shows the classical baseline (approximately 815,000 parameters, with roughly 24,000 in the bottleneck branch) achieves test AUC 0.862. The quantum hybrid (approximately 791,000 parameters, with only 54 quantum gate parameters in the bottleneck) achieves AUC 0.858. That is a roughly 450x reduction in bottleneck parameters with less than 0.5% AUC loss. During GSoC, I will compare three encoding strategies (amplitude, IQP, data re-uploading), benchmark each against a fixed (untrained) encoding baseline.

2. Background

2.1 Quantum Computing for HEP

Quantum computing on qubit-based platforms has the potential to improve the time complexity of classical machine learning algorithms by operating in exponentially large Hilbert spaces with relatively few physical parameters. Whether this potential translates into practical advantages for high energy physics (HEP) analysis is still an open question. The goal of this project, as stated by the ML4SCI QMLHEP group, is to explore and demonstrate that quantum computing can serve as a new paradigm for HEP data analysis as a proof of principle. Even on near-term Noisy Intermediate-Scale Quantum (NISQ) devices, which have limited qubit counts and coherence times, establishing that quantum representations can encode physics-relevant information without losing discriminating power would be a meaningful step. The preliminary results in this proposal suggest one concrete form this could take: a quantum circuit with 54 trainable parameters matching the discrimination performance of a classical network branch with roughly 24,000 parameters.

2.2 Jets and the Classification Task

At hadron colliders, quarks and gluons produced in collisions cannot exist freely due to color confinement and instead produce collimated sprays of hadrons called jets. Quark jets tend to be narrow (color factor $C_F = 4/3$) while gluon jets are broader with more soft radiation (color factor $C_A = 3$). Each particle is described by transverse momentum p_T (GeV), pseudorapidity η (related to polar angle by $\eta = -\ln(\tan(\theta/2))$), and azimuthal angle ϕ (rotation around the beam, periodic on $[-\pi, \pi]$). The ϕ periodicity requires $\text{atan2}(\sin, \cos)$ wrapping wherever ϕ

values are differenced. Here I use the Pythia8 Quark-Gluon dataset (100,000 jets, up to 139 particles each).

2.3 Contrastive Learning for Quantum Embeddings

A central challenge in quantum machine learning (QML) is the encoding of classical data into quantum states. Fixed encoding schemes, such as rotating each qubit by an angle proportional to one input feature, constrain the model before training even begins. Schuld, Sweke, and Meyer (2021) formalized this: a variational quantum model can be written as a partial Fourier series in the input data, and the frequencies accessible to the model are entirely determined by the encoding gates. If the encoding does not place the relevant frequencies within reach, the circuit cannot represent the decision boundary it needs, regardless of how many trainable parameters it has. This motivates learning the encoding from data rather than choosing it by hand.

Contrastive learning provides a framework for doing this without labeled data. The approach used here follows SimCLR (Chen et al., 2020): two randomly augmented views of the same jet are passed through the encoder, and the training objective (NT-Xent loss) pulls their representations together while pushing apart representations of different jets. The NT-Xent loss computes cosine similarities between all embedding pairs in the batch, scales them by a temperature parameter (set to 0.5), and treats finding the correct positive pair as a classification problem over the batch. No class labels (quark vs gluon) are used during this pretraining stage. When a variational quantum circuit sits at the output of the classical backbone, contrastive training forces the circuit to learn an encoding that captures the structure present in the data, rather than relying on a fixed mapping that may or may not be appropriate.

The claim here is not that the quantum circuit is faster or more accurate than the classical alternative. Rather, the question is whether the quantum model can achieve comparable discrimination with far fewer trainable parameters in the bottleneck (54 quantum gate parameters vs approximately 24,000 in the classical branch), which would indicate that the Hilbert space structure provides an efficient parameterization for this feature mapping task.

2.4 PointVector for Jets

Jets are unordered particle sets, i.e. point clouds. PointVector (Deng et al., CVPR 2023) processes them by hierarchically sampling representative points, grouping local neighborhoods via k-Nearest Neighbors (kNN), and aggregating features with directional attention that distinguishes displacement in eta from displacement in phi. This matters because the angular substructure distinguishing quark from gluon jets is inherently directional. In my prototype, PointVector compresses a 64-particle jet into a 256-dimensional embedding before the quantum layer.

3. Preliminary Work and Results

3.1 Architecture and Data Pipeline

The raw dataset is preprocessed by sorting particles by descending p_T , taking the top 64, centering in eta-phi space using p_T -weighted centroids, normalizing p_T to sum to 1, and computing $\log(p_T)$. The data splits 80/10/10 into train/validation/test with stratified sampling.

The PointVector encoder has three Vector Set Abstraction layers that downsample: 64 particles to 32 centroids (128 channels), then 16 (256 channels), then 8 (512 channels). Each layer uses

Farthest Point Sampling (FPS), kNN grouping ($k=16$), and directional vector aggregation. Global max pooling over the 8 final centroids gives a 512-dim vector, projected to 256 by a two-layer MLP (Multi-Layer Perceptron). A linear layer then compresses to 64 dimensions. The VQC uses amplitude embedding (mapping the 64-dim vector to 6-qubit amplitudes), ZXZ rotations (3 angles per qubit per layer), and CNOT gates in ring topology (qubit i controls qubit $(i+1) \bmod 6$) plus skip connections on even layers. Measurement via `qml.probs` returns all 64 basis state probabilities.

Augmentations: random phi rotation (a true detector symmetry), biased particle dropout (soft particles dropped more than hard ones, enforcing infrared safety), pT smearing (Gaussian multiplicative noise), and small eta-phi translations. The contrastive loss is NT-Xent with temperature 0.5. Evaluation: freeze the encoder, fit logistic regression on embeddings, report AUC-ROC.

3.2 Results

Configuration	Samples	Acc	AUC	Total Params	Bottleneck Params
Classical PointVector	80,000	0.783	0.862	~815,000	~24,000 (classical)
Quantum 6q / 2 layers	10,000	0.759	0.838	~791,000	54 (quantum)
Quantum 6q / 3 layers	40,000	0.774	0.855	~791,000	54 (quantum)
Quantum 6q / 3 layers	80,000	0.783	0.858	~791,000	54 (quantum)

The quantum model scales with data (0.838 AUC at 10k to 0.858 at 80k), suggesting data is the bottleneck, not capacity. Adding a third VQC layer helped (0.838 to 0.855 at matched data). At full data, the quantum model is 0.4% below the classical baseline while using a bottleneck that is roughly 450x smaller in parameter count. The practical lesson: amplitude embedding on `default.qubit` with PennyLane parameter broadcasting trains at about 8 minutes for 10k samples and 80 minutes for 80k on an A100 GPU. Phi-wrapping with `atan2` is essential everywhere. Batch indexing in the point cloud layers requires careful shape management when expanding for FPS and kNN.

4. Proposed Work

Two deliverables: (1) implement and train multiple embedding functions with contrastive learning, (2) benchmark trained embeddings against fixed (untrained) encodings on downstream classification.

4.1 Embedding Function Exploration

- **Amplitude Embedding (current baseline):** Maps 2^n features into n qubit amplitudes. Currently 6 qubits (64-dim). I will scale to 7 qubits (128-dim) and 8 qubits (256-dim, eliminating the compression layer entirely). Tradeoff: simulation cost scales exponentially with qubit count.
- **IQP Embedding:** Instantaneous Quantum Polynomial circuits use Hadamard gates on all qubits followed by diagonal ZZ interaction gates, repeated multiple times. These feature maps are provably hard to classically simulate under standard complexity assumptions. Maps n features to n qubits (one per qubit plus pairwise interactions), avoiding the amplitude normalization constraint. Available as `qml.IQPEmbedding` in PennyLane.

- **Data Re-uploading:** Instead of encoding data once at the start, re-uploading interleaves encoding gates with trainable variational gates across every circuit layer. Pérez-Salinas et al. (2020) showed a single qubit with re-uploading can approximate any function because each re-upload extends the accessible Fourier spectrum. I will re-encode the classical features via rotation gates at the start of each VQC layer, sandwiched between the trainable ZXZ rotations and CNOT entanglement.

The variational ansatz (ZXZ rotations + CNOT ring/skip entanglement) stays fixed across all experiments so differences in performance come from the encoding, not the processing circuit.

4.2 Loss Variants

I will compare NT-Xent (current, softmax-based) against a contrastive pair loss with fixed margin (directly optimizes Euclidean distance between positive/negative pairs). The pair loss is simpler and can be more stable at small batch sizes, which matters when quantum simulation limits batch size.

4.3 Benchmarking Against Fixed Encoding

For each trained embedding I will compare against a fixed-encoding baseline: same quantum circuit architecture, but with the encoding chosen a priori (no contrastive pretraining) and only the VQC classifier parameters trained. If contrastive-trained encodings consistently outperform fixed ones, that answers the central question of this project.

4.4 Stretch: JetClass Dataset

If time allows, I will move to the JetClass dataset (Qu et al., 2022): 10-class jet classification, 17 features per particle, 125M jets. The quark-gluon subset serves as a higher-quality replacement for Pythia8.

5. Timeline

Period	Tasks	Deliverables
Bonding (May 26–Jun 8)	Set up environment. Refactor prototype into modular codebase. Read mentor papers. Align on priorities.	Modular repo, tracking setup
Wk 1–2 (Jun 9–22)	Implement IQP and data re-uploading embeddings. Initial training on Pythia8 for all three encodings with matched hyperparameters.	3 trained embeddings + AUCs
Wk 3–4 (Jun 23–Jul 6)	Sweep qubits (4, 6, 8), depth (1–4 layers), temperature, LR. Systematic comparison.	Sweep results, scaling analysis
Wk 5–6 (Jul 7–20)	Pair loss variant. Train all embedding×loss combos. Begin fixed-encoding benchmark.	Loss comparison, initial benchmarks
Midterm (Jul 21–25)	Midterm report and lightning talk.	Report + presentation
Wk 7–8 (Jul 28–Aug 10)	Complete benchmarking: downstream VQC classifiers on trained vs fixed encodings. Multiple seeds for error bars.	Benchmark tables + ROC curves
Wk 9–10 (Aug 11–24)	Ablations (augmentations, projection head, data volume). t-SNE/UMAP embedding visualizations. JetClass if time.	Ablations, visualizations
Wk 11–12 (Aug 25–Sep 7)	Code cleanup, docs, README. Final blog for ML4SCI. Final report and talk. Colab reproducibility.	Blog, codebase, final report

I am applying for the 175-hour track (approximately 14 hours/week over 12 weeks). The core deliverables (Tasks 1-3: embedding exploration, loss variants, and benchmarking against fixed encoding) fit within this scope. The JetClass extension (Task 4.4) is a stretch goal.

6. Background and Experience

- **Quantum physics + ML:** Prior hackathon project predicting quantum spin interaction parameters from simulated NMR (Nuclear Magnetic Resonance) magnetization curves, using ML on Gaussian and RKKY (Ruderman-Kittel-Kasuya-Yosida) coupling models.
- **Point cloud architectures:** Built PointVector from scratch, including FPS, kNN grouping, and directional vector aggregation. Debugged batch indexing shape mismatches in production.
- **PennyLane:** Implemented custom VQCs with amplitude embedding and ZZ rotations. Handled device compatibility (default.qubit vs lightning.qubit, backprop vs adjoint). Used parameter broadcasting for batched simulation.
- **Jet physics:** Understand color confinement, hadronization, detector coordinates (pT, eta, phi), and the radiation pattern differences that drive quark-gluon discrimination.

Repository: github.com/aviicharya

7. Why This Project

The question is concrete and testable: does learning the data encoding improve QML for HEP? My preliminary evidence says yes, at least in terms of parameter efficiency. But the prototype only tested one encoding and one loss. A proper comparison across encodings and losses, benchmarked against a fixed-encoding baseline, would be much more convincing. The mentor team spans QML methodology (Wisconsin-Madison), experimental HEP (Alabama), and quantum computing (TUM), which is the right combination of expertise for this work.

8. References

- [1] Schuld, Sweke, Meyer (2021). *Effect of data encoding on expressivity of variational QML models*. Phys. Rev. A 103.
- [2] Chen et al. (2020). *SimCLR: A Simple Framework for Contrastive Learning*. ICML.
- [3] Deng et al. (2023). *PointVector: A Vector Representation in Point Cloud Analysis*. CVPR.
- [4] Qu et al. (2022). *Particle Transformer for Jet Tagging*. ICML.
- [5] Pérez-Salinas et al. (2020). *Data re-uploading for a universal quantum classifier*. Quantum 4:226.
- [6] Nanda (2024). GSoC 2024: *Quantum contrastive learning for HEP*. ML4SCI/QMLHEP.
- [7] Jahin et al. (2024). *Quantum Rationale-Aware Graph Contrastive Learning for Jet Discrimination*. arXiv:2411.01642.

9. Availability

14 hours/week, 175-hour track. No summer conflicts. Available for weekly mentor calls and ML4SCI meetings. Timezone: US Central (CDT).